Full Stack Development with MERN

Project Documentation

1. Introduction

- **Project Title:** ShopEZ Ecommerce Application
- **Team Members:** Ruturaj Misal (Individual)
- **GitHub Repository:** https://github.com/ruturaj1804/-ShopEZ

2. Project Overview

• Purpose:

ShopEZ Application is a modern, responsive full-stack e-commerce web platform engineered using the MERN stack (MongoDB, Express.js, React 18, Node.js). Designed for everyday online retail, it provides consumers with a frictionless shopping journey—from product catalog exploration, keyword search, and category filtering to persistent shopping cart synchronization, transparent reviews, and Cash on Delivery (COD) checkout. Simultaneously, it equips store merchants and administrators with a dedicated Seller Central portal featuring real-time inventory tracking, inline price/stock updates without page reloads, automated low-stock warnings (< 10 units), and gross revenue analytics.

• Features:

- **User Authentication & Authorization:** Registration, secure login, stateless JWT bearer token authentication, role-based access control (`user` / `admin`), and client/server protected route wrappers.
- **Interactive Storefront:** High-performance landing banner, category filtering (Electronics, Clothing, Footwear, Home, Books), instant regex search, and multi-parameter price sorting.
- **Product Detail & Customer Reviews:** High-resolution product images, discount tags, dynamic stock indicators, and customer rating/review submission with 1–5 stars.
- **Persistent Shopping Cart:** Real-time cart calculations, quantity increment/decrement, line-item subtotals, and instant badge updates in the top navigation bar.
- **Streamlined Checkout & Order Tracking:** Seamless shipping address entry, Cash on Delivery (COD) checkout, order document creation, automatic stock deduction, and real-time status tracking (`Processing` → `Shipped` → `Delivered` → `Cancelled`).
- **Seller Central & Operations:** Dedicated `/admin/sell` dashboard with sales summaries, inline editable table cells for prices and stock counts, and automated visual low-stock warning banners when inventory drops below 10 units.
- **Admin Dashboard:** Centralized hub displaying gross revenue analytics, order volume, total product listings, and registered customer governance.
- **Security & Engineering Controls:** Passwords salted with `bcryptjs` (10 rounds), CORS allowlists, centralized Express error handling, and file uploads powered by Multer.

3. Architecture

The application follows a strictly decoupled 3-tier client/server architecture. The React single-page application (SPA) communicates asynchronously with an Express.js backend via RESTful JSON endpoints. Requests are validated through custom middleware before reaching controllers that interact with MongoDB via Mongoose ODM schemas.

ShopEZ Markets — System Architecture

A secure, scalable, and modular 3-tier full-stack architecture for modern e-commerce operations.

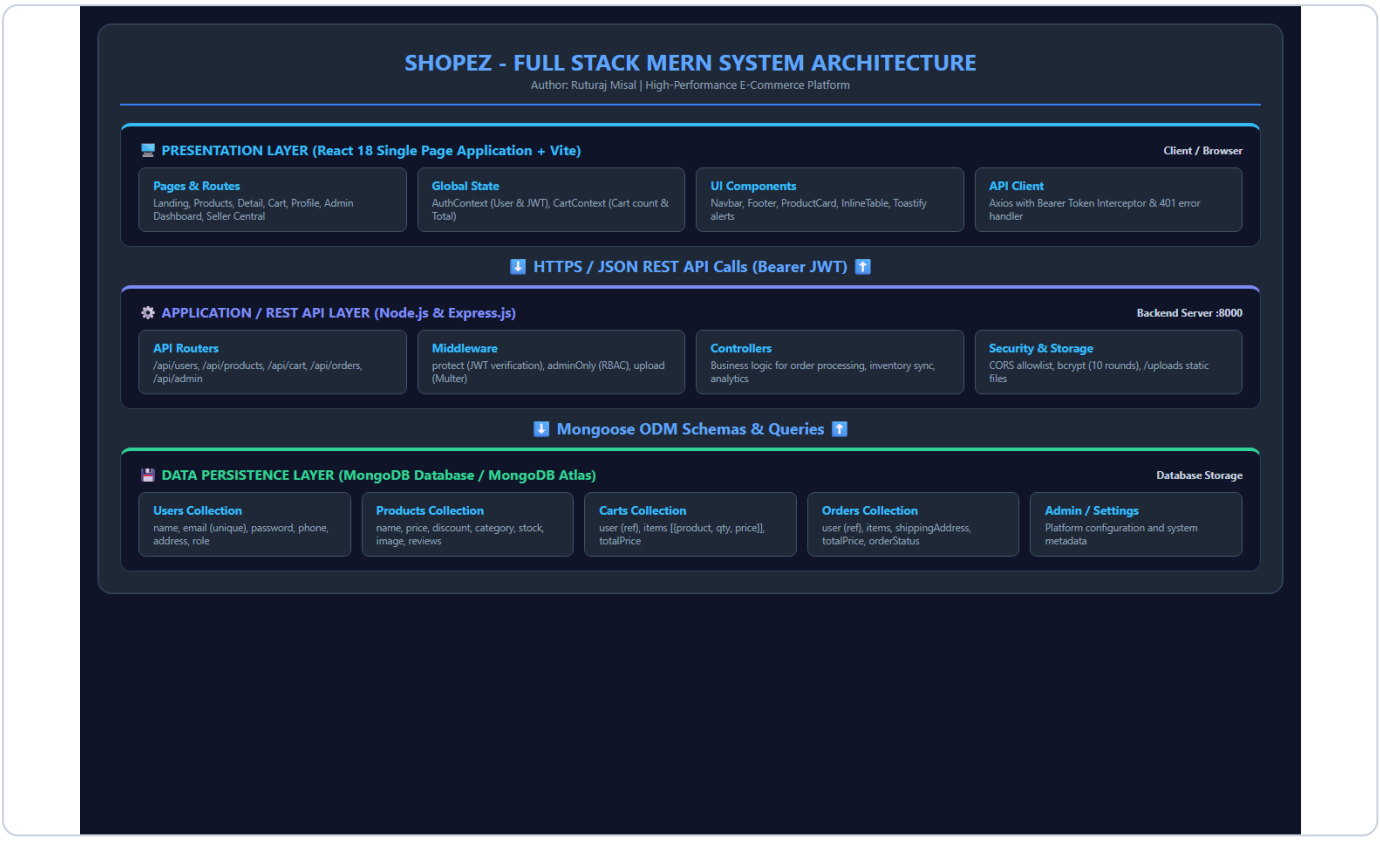


Figure 1: Full-Stack MERN System Architecture (Client, API Server & Database Layers)

Frontend Architecture (React 18 + Vite)

The frontend is built with React 18 and Vite for near-instant build times and Hot Module Replacement (HMR). Routing is structured through React Router DOM (v7) with declarative ProtectedRoute wrappers. Global application state is handled elegantly using the React Context API:

- **AuthContext:** Manages user credentials, authentication status, and JWT persistence in `localStorage`. Provides seamless login, logout, and automatic session restoration.
- **CartContext:** Manages cart line items, dynamic quantity adjustments, total pricing, and synchronization with backend Cart endpoints.
- **Axios Client:** Custom HTTP client configured with request interceptors to automatically attach the `Authorization: Bearer <token>` header and response interceptors to catch 401 unauthenticated errors.

Backend Architecture (Node.js + Express.js)

The backend REST API server follows the classic Model-View-Controller (MVC) architectural pattern. Server entry point (`index.js`) configures global middleware (CORS, JSON body parser, URL encoding, static serving of the `/uploads` directory), mounts API routes, and establishes database persistence.

- **Routes:** Declarative modular routing (`userRoutes`, `productRoutes`, `cartRoutes`, `orderRoutes`, `adminRoutes`).
- **Middleware:** `protect` validates JWT tokens; `adminOnly` enforces administrative privileges; `upload` processes multipart image uploads via Multer.
- **Controllers:** Encapsulate core business logic: user registration, password verification, inventory manipulation, dynamic cart math, order status transitions, and revenue aggregations.

Database Schema & Interactions with MongoDB

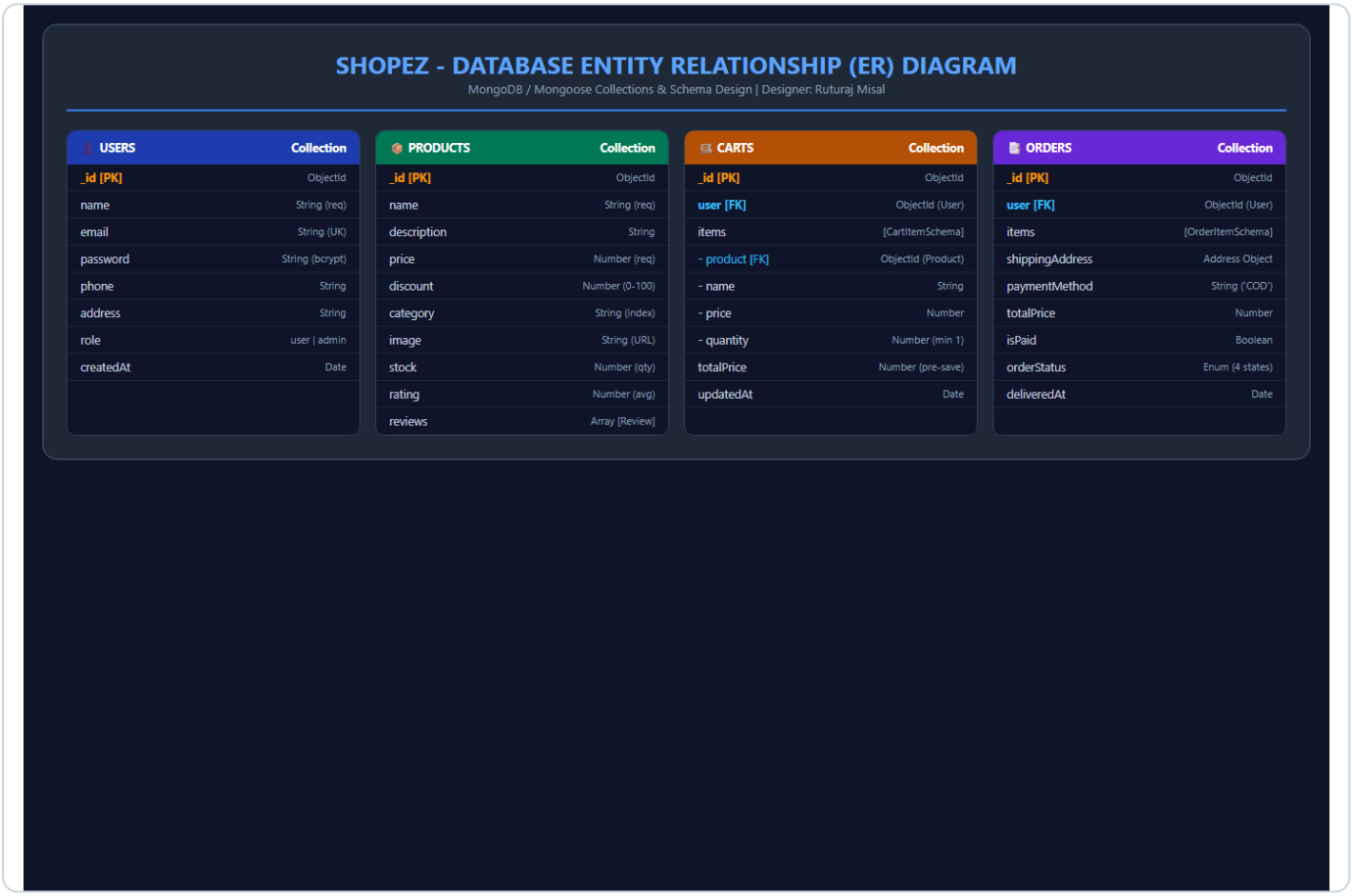


Figure 2: MongoDB Entity Relationship (ER) Schema & Collection Design

4. Setup Instructions

• Prerequisites:

- Node.js (v16+ or v18+ recommended) and npm
- MongoDB locally running on `localhost:27017` or a MongoDB Atlas cloud URI
- A modern web browser (Google Chrome, Microsoft Edge, Firefox)

• Environment Variables:

File	Variable	Purpose & Default Value
`Server/.env`	`PORT`	Express server port (default `8000`)
`Server/.env`	`MONGO_URI`	MongoDB connection string (e.g., `mongodb://localhost:27017/shopez`)
`Server/.env`	`JWT_SECRET`	Secret cryptographic key used for signing JWTs
`Server/.env`	`NODE_ENV`	Runtime environment (`development` or `production`)
`Client/.env`	`VITE_API_URL`	Backend API base URL (default `http://localhost:8000/api`)

5. Folder Structure

```
ShopEZ E-commerce Application/ ├── Documentation/ │ ├── Project Documentation.pdf # Official formatted project documentation │ ├── Project_Documentation.md # Comprehensive markdown documentation │ └── diagrams/ # High-resolution architectural graphics & UI mockups ├── Phase Wise/ # Complete 5-phase engineering deliverables │ ├── Brainstorming & Ideation Phase/ # Problem statements, empathy canvas & idea prioritization │ ├── Project Planning Phase/ # Product backlog, sprint schedules & estimation │ ├── Requirement Analysis/ # Functional/NFR requirements, tech stack & DFDs │ ├── Project Design Phase/ # Architecture, proposed solution & problem-solution fit │ └── Project Developement/ # User Acceptance Testing (UAT) & test matrix ├── Client/ # React 18 frontend (Vite) │ ├── src/components/ # Navbar, Footer, ProductCard │ ├── src/context/ # AuthContext, CartContext │ ├── src/pages/ # Landing, Products, Cart, Profile, Login, Register │ └── src/pages/admin/ # AdminDashboard, SellerDashboard, AllOrders, NewProduct └── Server/ # Node.js & Express.js backend ├── config/ # MongoDB connection (db.js) ├── controllers/ # Business controllers (user, product, cart, order, admin) ├── middleware/ # JWT auth & Multer image upload ├── models/ # Mongoose schemas (User, Product, Cart, Order, Admin) ├── routes/ # Express API route endpoints └── uploads/ # Product image assets directory
```

6. Running the Application

```
# 1. Start the Backend Server (Runs on http://localhost:8000) cd Server npm install node seed.js # Seeds initial products, categories, and admin account npm run dev # Starts Express server via nodemon # 2. Start the Frontend Client (Runs on http://localhost:5173) cd ../Client npm install npm run dev # Launches Vite development server with HMR
```

7. API Documentation

All endpoints are prefixed with `/api`. Protected routes require an `Authorization: Bearer <JWT>` header.

Method	Endpoint	Access	Description
GET	/api/health	Public	Server health check and uptime status
POST	/api/users/register	Public	Register a new user: name, email, password, phone, address
POST	/api/users/login	Public	Authenticate user and return JWT bearer token
GET	/api/users/profile	User	Fetch profile information of logged-in user
PUT	/api/users/profile	User	Update personal profile (name, phone, address)
GET	/api/products	Public	Fetch catalog (supports search, category, sort)
GET	/api/products/:id	Public	Retrieve single product details and customer reviews
POST	/api/products	Admin	Create product listing with image upload (Multer multipart)
PUT	/api/products/:id	Admin	Update product (supports inline price & stock updates)
DELETE	/api/products/:id	Admin	Delete product from inventory
POST	/api/products/:id/reviews	User	Submit review with star rating (1–5) and comment
GET	/api/cart	User	Fetch user's shopping cart and recalculated total price
POST	/api/cart/add	User	Add product item to cart (payload: productId, quantity)
PUT	/api/cart/:itemId	User	Update item quantity in cart
DELETE	/api/cart/:itemId	User	Remove single item from cart
DELETE	/api/cart/clear/all	User	Empty entire shopping cart
POST	/api/orders	User	Place new order with shipping address & COD payment
GET	/api/orders/myorders	User	Retrieve all orders placed by the current user
GET	/api/orders/all	Admin	Retrieve all customer orders across the platform
PUT	/api/orders/:id/status	Admin	Update order status (Processing, Shipped, Delivered)
GET	/api/admin/dashboard	Admin	Fetch aggregate metrics: revenue, orders, products, users

Sample API Payloads

1. Registration (POST /api/users/register)

```
POST /api/users/register { "name": "Ruturaj Misal", "email": "ruturaj@example.com", "password": "SecurePassword123", "phone": "+91 9876543210", "address": "123 Innovation Park, High-Tech City, Pune, MH" } // Response 201 Created: { "success": true, "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...", "user": { "_id": "64f1a2b...", "name": "Ruturaj Misal", "email": "ruturaj@example.com", "role": "user" } }
```

2. Product Catalog Query (GET /api/products?category=Electronics&sort=price_asc)

```
GET /api/products?category=Electronics&sort=price_asc // Response 200 OK: { "success": true, "count": 2, "data": [ { "_id": "64f1c9d...", "name": "Wireless Noise-Canceling Headphones", "price": 2499, "discount": 20, "category": "Electronics", "stock": 25, "rating": 4.8 }, { "_id": "64f1c9e...", "name": "Portable Bluetooth Speaker", "price": 1499, "discount": 25, "category": "Electronics", "stock": 4, "rating": 4.5 } ] }
```

3. Place Order (POST /api/orders)

```
POST /api/orders Authorization: Bearer <JWT> { "items": [{ "product": "64f1c9d...", "name": "Wireless Headphones", "price": 2499, "quantity": 1 }], "shippingAddress": { "address": "123 Innovation Park", "city": "Pune", "state": "MH", "zipCode": "411045", "country": "India" }, "paymentMethod": "COD", "totalPrice": 2499 } // Response 201 Created: { "success": true, "message": "Order placed successfully", "data": { "_id": "64f1e5a...", "orderStatus": "Processing", "totalPrice": 2499, "isPaid": false } }
```

8. Authentication & Security Architecture

Authentication is completely stateless, powered by industry-standard JSON Web Tokens (JWT). Upon successful login or registration, the server signs a cryptographically secure token containing the user's primary key (`id`) and access role (`user` or `admin`).

- **Password Security:** Passwords are encrypted before database persistence using `bcryptjs` with 10 salt rounds. Plaintext passwords are never stored or logged.
- **Route Protection (`protect`):** Decodes the Bearer token in the `Authorization` header, verifies signature using `JWT_SECRET`, and attaches the active user document to `req.user`.
- **Role Authorization (`adminOnly`):** Restricts critical operational routes (product additions, inline stock updates, platform analytics) strictly to administrative users.
- **Automatic Session Eviction:** Axios interceptors intercept HTTP 401 unauthenticated errors, clearing cached localStorage credentials and redirecting users to the login route.

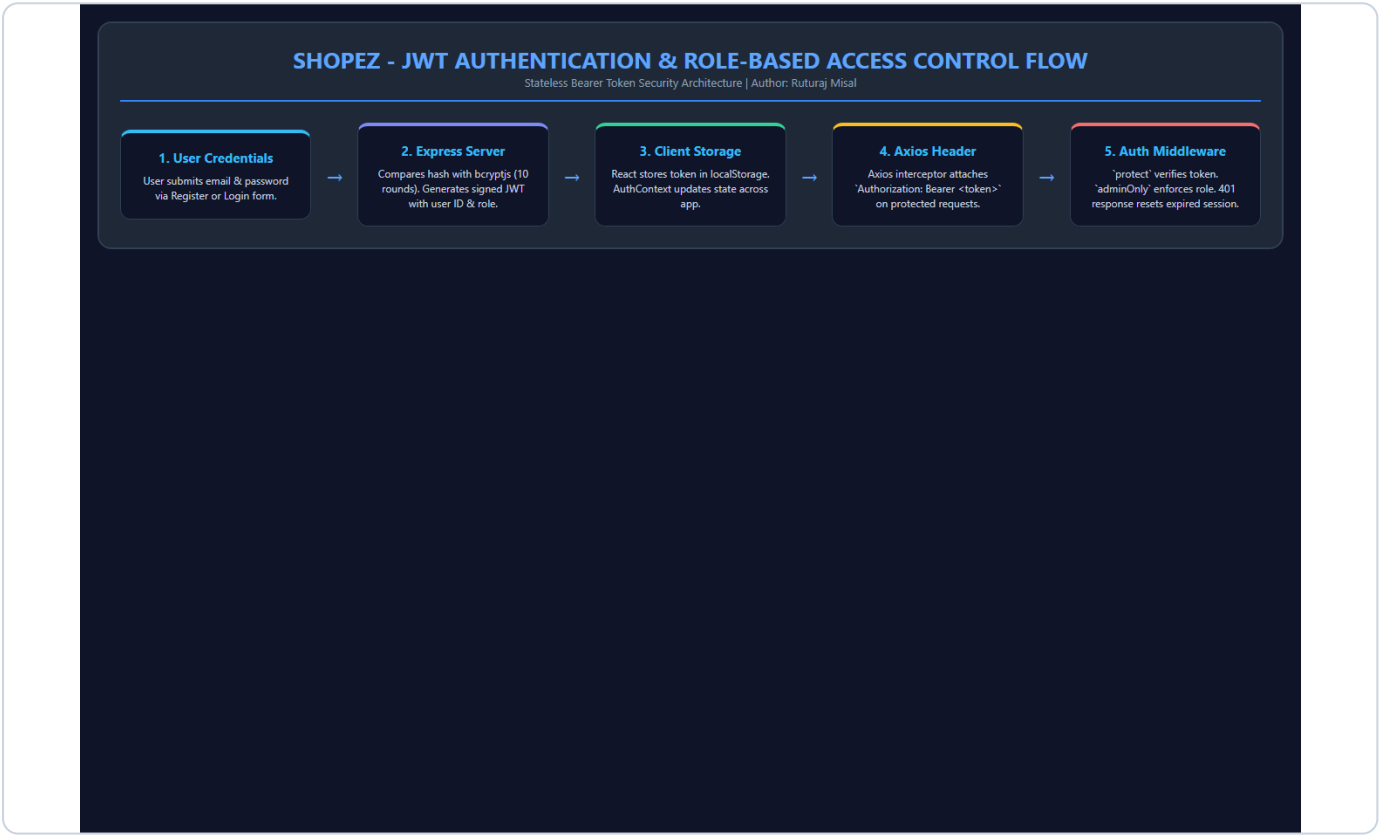


Figure 3: ShopEZ JWT Authentication, Token Persistence & Role-Based Guard Flow

9. Testing Strategy & Quality Assurance

ShopEZ adheres to a multi-tiered testing strategy encompassing Unit Testing, Integration Testing, and comprehensive User Acceptance Testing (UAT).



Figure 4: Testing Pyramid and Verification Areas for ShopEZ

- **Sample UAT Verification Results:**

Test ID	Scenario	Test Steps & Verification	Status
TC-001	User Registration	Enter name, email, password → JWT generated, profile created in MongoDB	PASS
TC-002	User Login	Submit valid credentials → Token saved to localStorage, navbar updates	PASS
TC-004	Catalog Search & Filter	Filter by "Electronics", search "Wireless" → Instant regex query execution	PASS
TC-006	Shopping Cart Update	Add item to cart → Dynamic subtotal recomputed, cart saved in DB	PASS
TC-008	Checkout (COD)	Submit shipping address → Order created, product stock decremented	PASS
TC-012	Inline Inventory Edit	Change stock count in Seller Dashboard → Immediate persistence in MongoDB	PASS
TC-013	Low Stock Alert	Set stock to 4 units → Visual warning banner (< 10 units) appears	PASS

10. User Interface — Home / Landing Page

The landing page welcomes shoppers with an engaging hero banner, category discovery pills (Electronics, Clothing, Footwear, Home, Books), and featured merchandise cards with discount badges and star ratings.

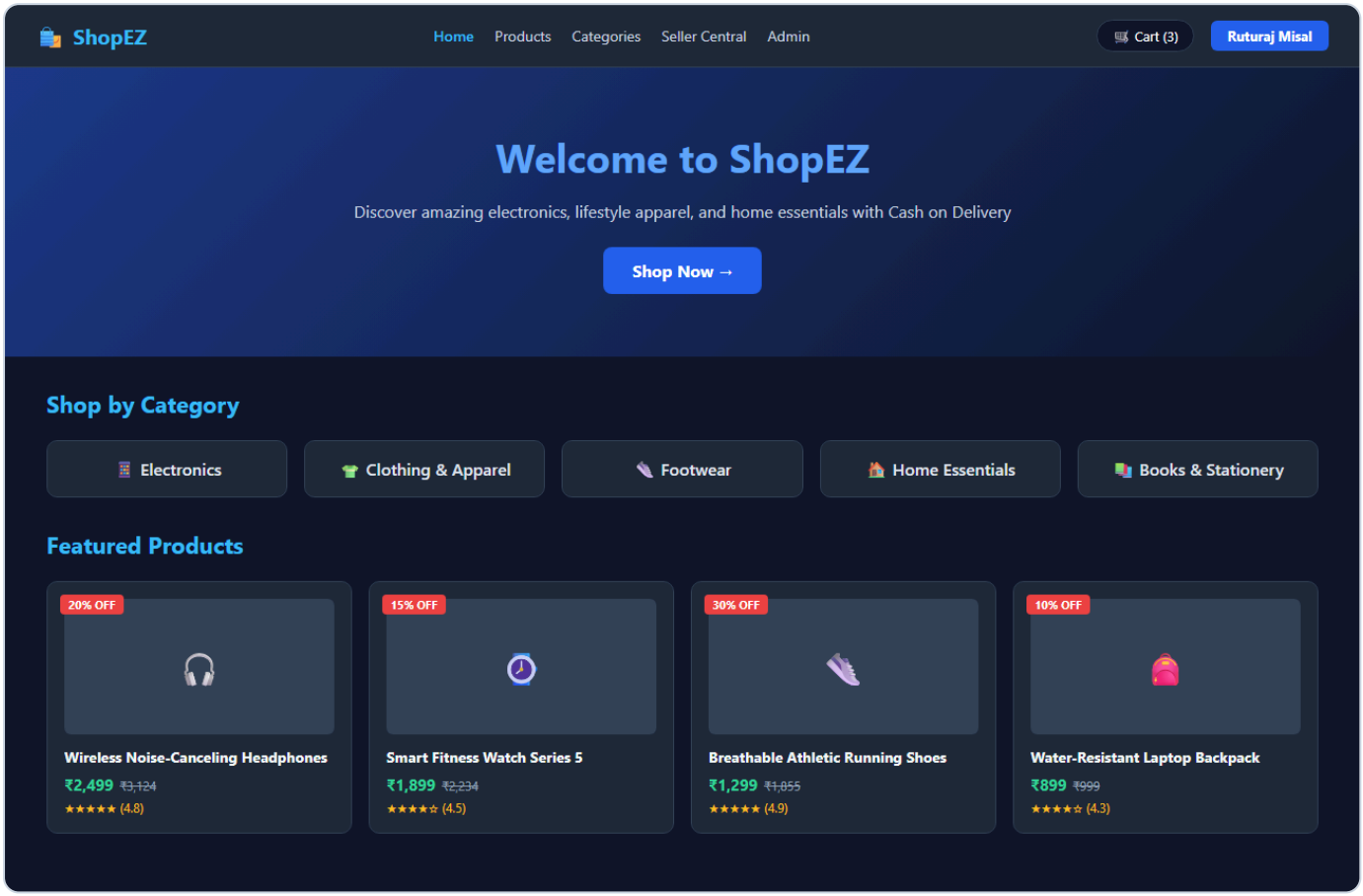


Figure 5: ShopEZ Storefront Landing Page with Hero Banner & Featured Product Grid

User Interface — Product Catalog & Filtering

The Products page allows users to explore the complete catalog with live keyword searching, price range filtering, category tabs, and real-time stock availability indicators.

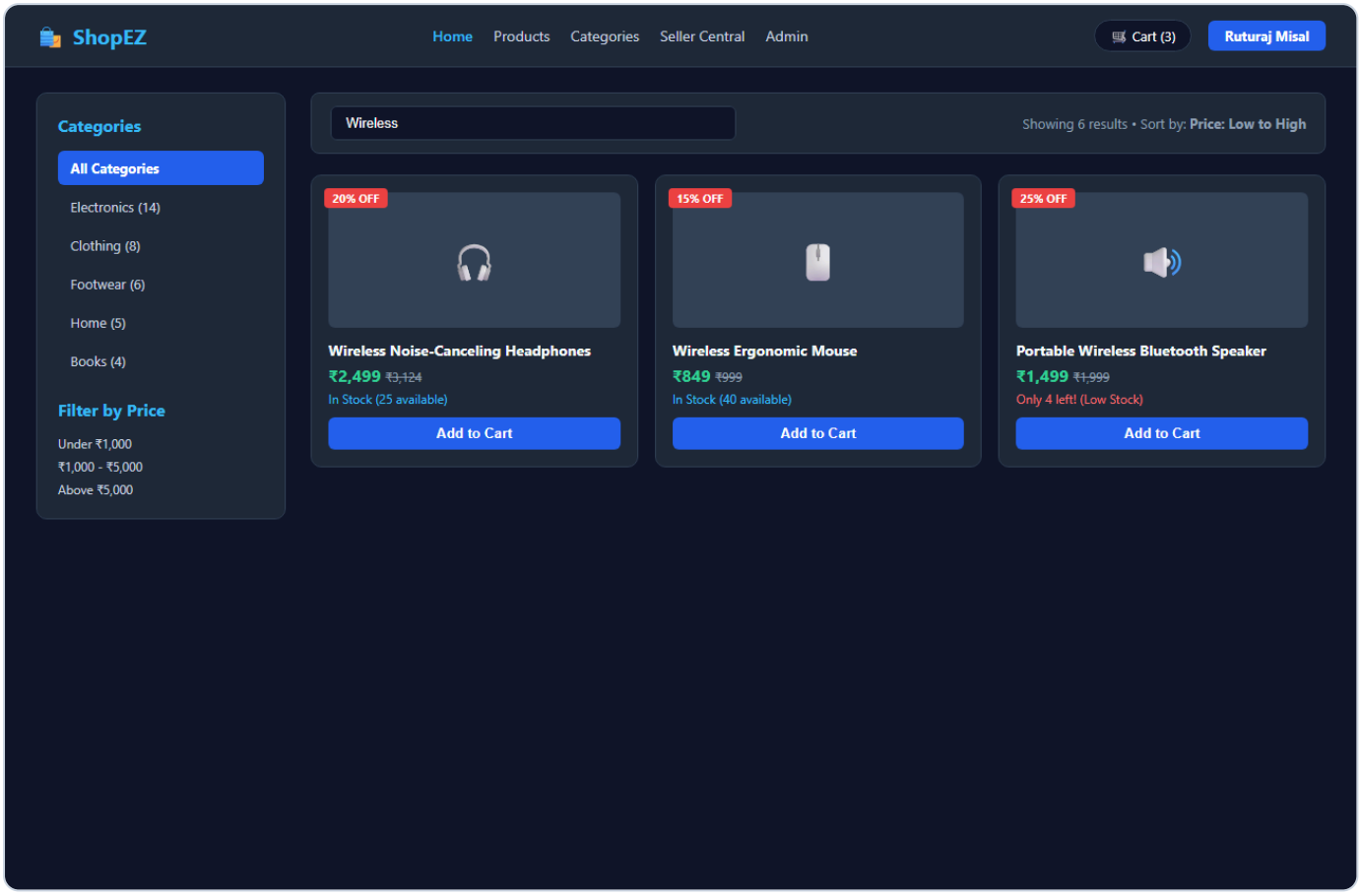


Figure 6: Interactive Product Catalog with Search Bar & Category Navigation

User Interface — Shopping Cart & Price Recalculation

The shopping cart features instant line-item quantity manipulation (`[-] [qty] [+]`), item removal, and an automatic order summary card computing taxes, discounts, and total cost before checkout.

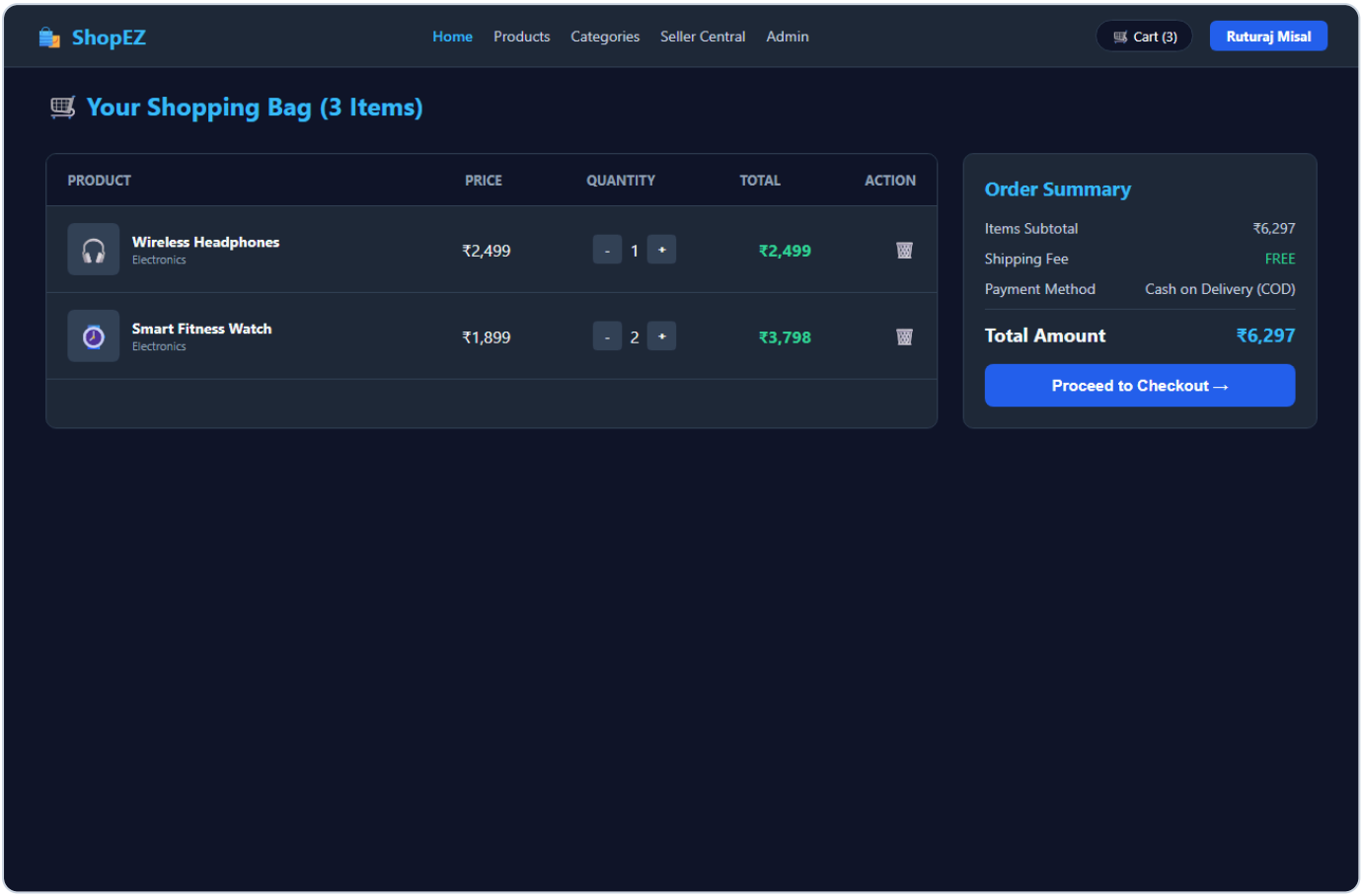


Figure 7: Shopping Bag Interface with Dynamic Line-Item & Total Price Recalculations

User Interface — User Profile & Order History

The profile dashboard provides users with quick access to default shipping addresses and a complete historical ledger of all past orders with status badges (`Processing`, `Shipped`, `Delivered`).

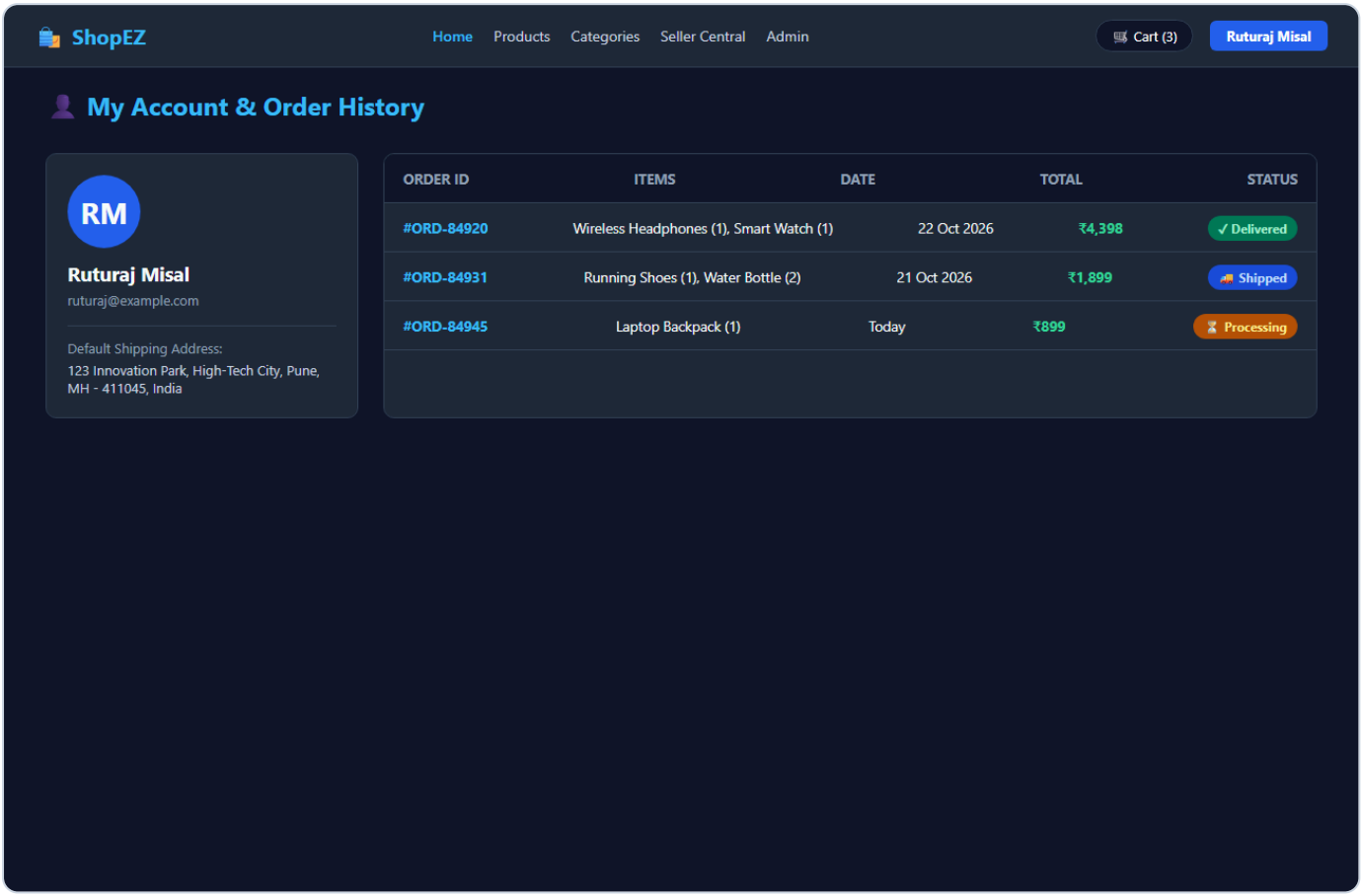


Figure 8: Customer Profile Dashboard with Real-Time Order History & Tracking

User Interface — Seller Central Dashboard (/admin/sell)

Designed for rapid merchant operations, the Seller Dashboard includes gross sales overview cards, automated low-stock warnings (< 10 units), and inline editable table cells for immediate catalog adjustments.

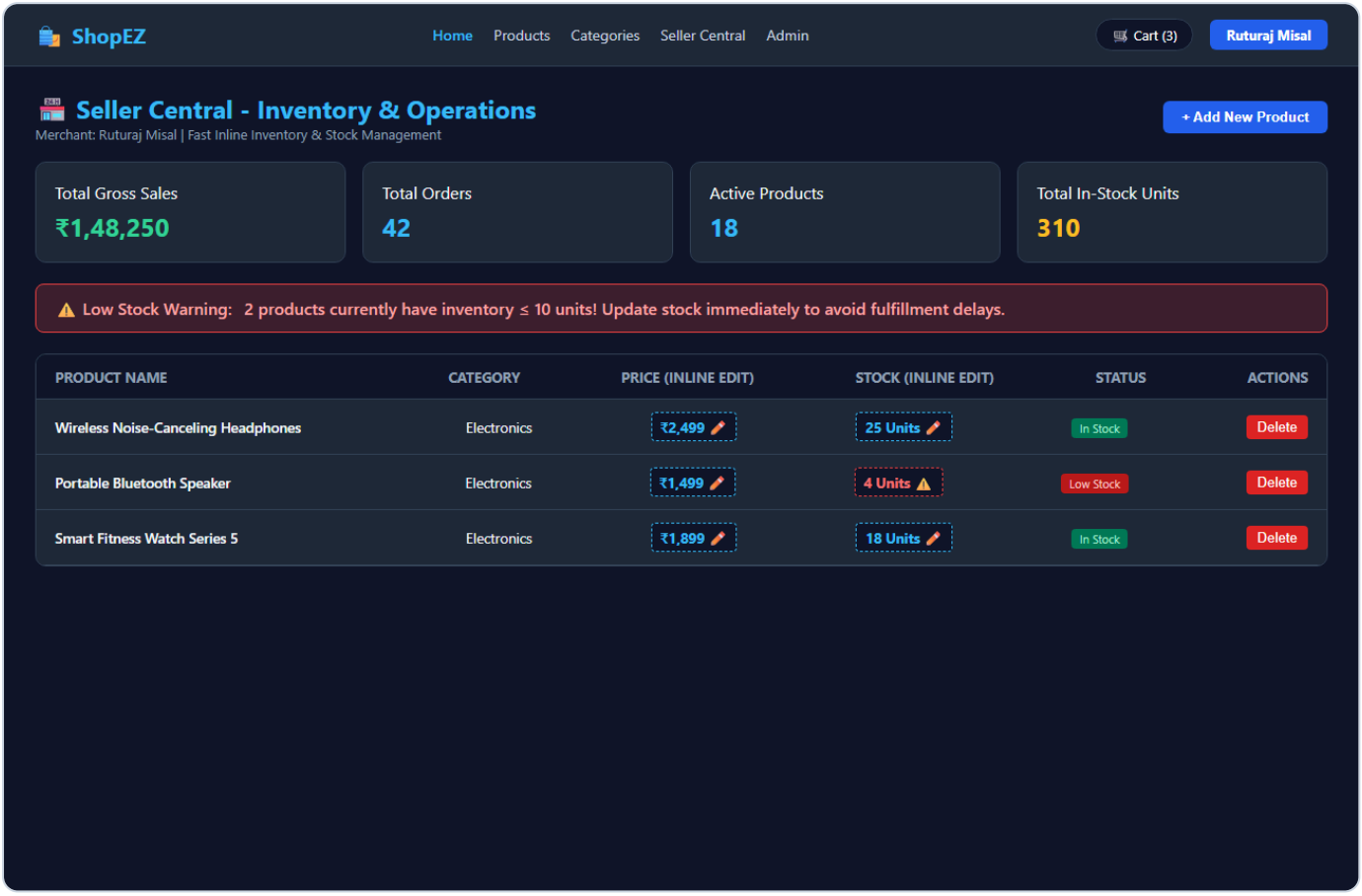


Figure 9: Seller Central Dashboard with Inline Price/Stock Editing & Low-Stock Alerts

Data Flow Diagrams (DFD Level 0, 1 & 2)

Visualizing information flow across customers, merchants, controllers, and MongoDB data stores.

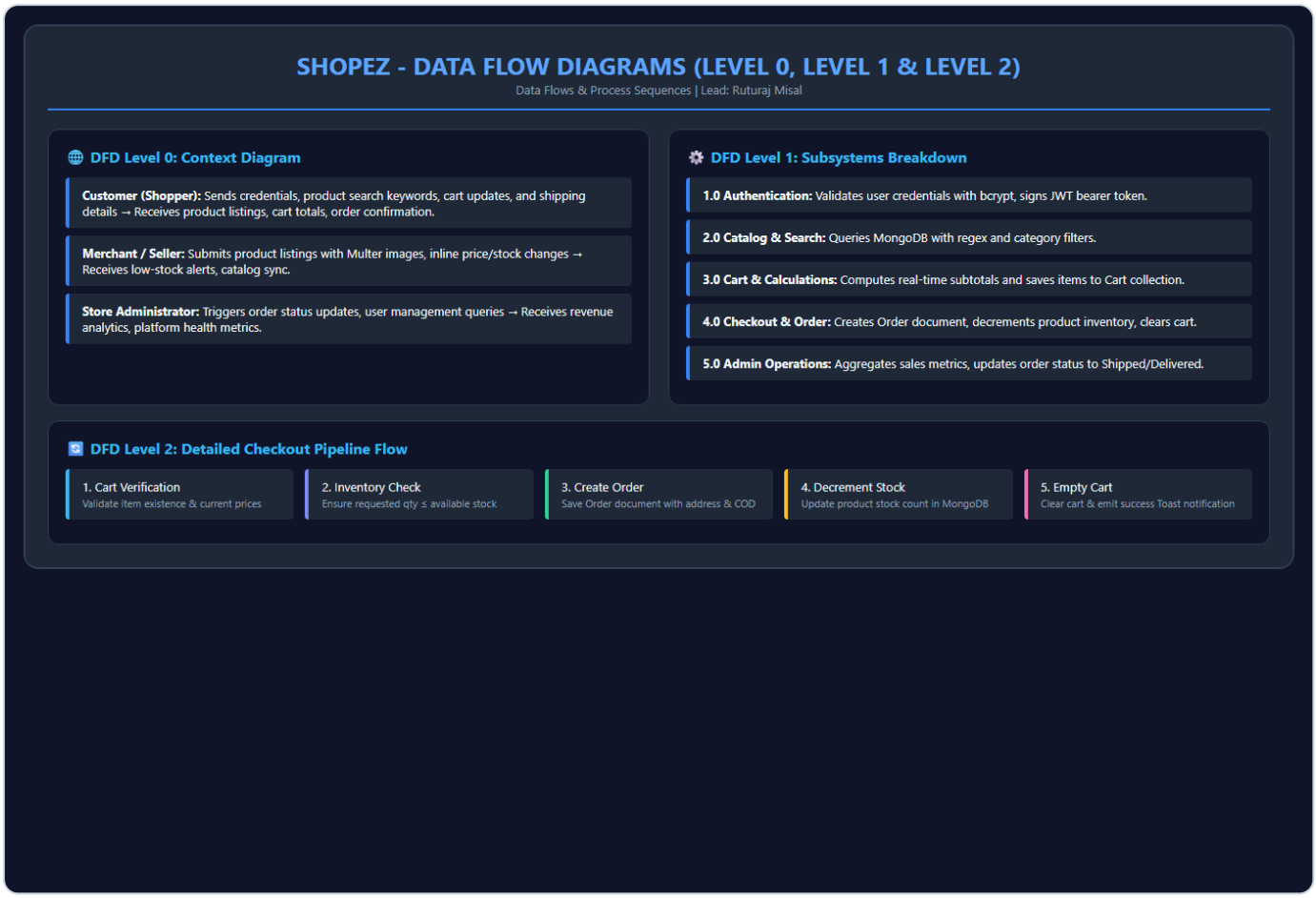


Figure 10: Multi-Level Data Flow Diagrams for ShopEZ Subsystems & Checkout Pipeline

Ideation Phase — Empathy Map & Problem Statement

Understanding shopper and merchant psychology to deliver an e-commerce platform that reduces friction.



Figure 11: Empathy Map Canvas for Online Consumers and Store Merchants

Ideation Phase — Action Priority Matrix (Impact vs Effort)

Strategic feature roadmap balancing immediate user value against implementation complexity.



Figure 12: Action Priority Matrix (Quick Wins, Major Projects, Fill-Ins & Future Iterations)

11. Known Limitations & Constraints

- **Ephemeral Local Uploads:** When deployed in serverless hosting environments like Vercel, static files saved in `/uploads` are ephemeral. Cloud storage services like Cloudinary or AWS S3 are recommended for persistent cloud image hosting.
- **Simulated Payment Gateway:** Order placement is fully functional using Cash on Delivery (COD). Digital online payments (Razorpay / Stripe) will be integrated in the upcoming Phase 2 production release.
- **Database Connection on Cloud:** Local connection strings (`mongodb://localhost:27017`) must be replaced with a MongoDB Atlas cloud URI when deployed to external cloud hosting environments.

12. Future Enhancements

- **Integrated Payment Gateway:** Native integration with Stripe and Razorpay for credit/debit card, UPI, and net banking transactions.
- **Cloudinary Image Integration:** Direct image uploads to Cloudinary CDN with automatic resizing, WebP compression, and fast global delivery.
- **Automated Email & SMS Notifications:** Automated order confirmation, dispatch alerts, and delivery receipts powered by Nodemailer or SendGrid.
- **Wishlist & Save for Later:** Allow shoppers to bookmark favorite products for future purchase without cluttering the active shopping bag.
- **AI Recommendation Engine:** Personalized product suggestions based on user browsing history and frequently co-purchased items.
- **Two-Factor Authentication (2FA):** Enhanced security for seller and administrator accounts via TOTP-based authentication.

13. Project Links & Verification

- **Author:** Ruturaj Misal (Individual)
- **GitHub Repository:** https://github.com/ruturaj1804/-ShopEZ
- **Documentation Status:** Approved & Verified (17 Pages with High-Resolution Architecture & UI Diagrams)